

Ingeniería del Software 1 — Clase 3

Material de Apoyo: La Agilidad del Proceso

Bloque 2 — Unidad 3 del Programa (Ecom-82): Desarrollo Ágil

1. De los modelos prescriptivos a la agilidad

En la Clase 2 estudiamos los modelos de proceso prescriptivos: cascada, incremental, evolutivo y concurrente. Todos comparten una lógica de planificación anticipada y documentación extensa, pensada para proyectos donde los requerimientos están razonablemente estables desde el inicio.

La pregunta que abre esta clase es: ¿qué pasa cuando los requerimientos cambian todo el tiempo? La respuesta de la industria del software, desde fines de los años 90, fue el desarrollo ágil: un conjunto de valores, principios y prácticas que aceptan el cambio como una condición normal del desarrollo de software, y no como una excepción a evitar.

2. ¿Qué es la agilidad?

La agilidad, en el contexto de la ingeniería de software, no es sinónimo de "ausencia de proceso" ni de "trabajar más rápido y sin documentar". Es la capacidad de un equipo de responder eficazmente al cambio, manteniendo el enfoque en la entrega continua de valor al cliente.

Un equipo ágil reconoce que el software es desarrollado por personas que trabajan en equipos, y que la efectividad de esos equipos depende tanto de factores técnicos como humanos: comunicación, confianza y capacidad de autoorganización.

3. La agilidad y el costo del cambio

En los modelos prescriptivos, el costo de modificar un requerimiento crece exponencialmente cuanto más avanzado está el proyecto: un cambio detectado en la fase de prueba es mucho más caro que uno detectado en el análisis.

Los procesos ágiles buscan aplanar esa curva: mediante iteraciones cortas, entregas frecuentes y retroalimentación constante del cliente, el cambio se detecta e incorpora antes de que se vuelva costoso. No es que el cambio "no cueste" en un proceso ágil — es que el proceso está diseñado para que ese costo se mantenga bajo y estable a lo largo de todo el proyecto.

4. ¿Qué es un proceso ágil?

4.1 Principios de agilidad

El Manifiesto Ágil (2001) resume la filosofía en cuatro valores centrales:

- Individuos e interacciones, por encima de procesos y herramientas.
- Software funcionando, por encima de documentación extensiva.
- Colaboración con el cliente, por encima de la negociación contractual.

- Respuesta ante el cambio, por encima de seguir un plan.

Es importante subrayar: el manifiesto no descarta los elementos de la derecha (procesos, documentación, contratos, planes) — reconoce que existen y tienen valor, pero prioriza los de la izquierda cuando hay que tomar una decisión.

4.2 La política del desarrollo ágil

Un proceso ágil se sostiene sobre un conjunto de políticas o compromisos organizacionales: iteraciones cortas y con entregables reales, participación constante del cliente, equipos pequeños y multidisciplinarios, simplicidad como estrategia de diseño, y disposición a modificar el rumbo del proyecto entre iteración e iteración.

4.3 Factores humanos

Pressman insiste en que la agilidad depende más de las personas que de las herramientas. Un equipo ágil efectivo necesita:

- Competencia técnica y capacidad de comunicación de cada integrante.
- Autoridad para tomar decisiones técnicas y de gestión sobre su propio trabajo.
- Confianza mutua entre los miembros del equipo y con el cliente.
- Capacidad de autoorganización, sin depender de una jerarquía rígida.

5. Programación Extrema (XP)

5.1 Valores XP

- **Comunicación:** el conocimiento del proyecto debe circular constantemente entre todo el equipo.
- **Simplicidad:** diseñar solo lo necesario para el requerimiento actual, no para hipotéticos futuros.
- **Retroalimentación:** obtener respuesta concreta y temprana sobre el software construido, tanto del sistema como del cliente.
- **Coraje:** tomar decisiones difíciles (por ejemplo, refactorizar código o descartar un enfoque) cuando la situación lo exige.
- **Respeto:** cada integrante del equipo valora el trabajo de los demás como condición de la colaboración.

5.2 El proceso XP

El ciclo de XP se organiza en torno a historias de usuario, iteraciones cortas y prácticas técnicas concretas:

- Planning game: negociación entre cliente y equipo sobre qué historias de usuario se abordan en cada iteración.
- Entregas pequeñas y frecuentes de software funcionando.
- Diseño simple, evitando la complejidad anticipada.
- Pruebas continuas, escritas antes o junto con el código (test-first).
- Refactorización constante del código, sin alterar su comportamiento externo.
- Programación en pares (dos personas, un puesto de trabajo).
- Integración continua del código de todo el equipo.
- Propiedad colectiva del código: cualquiera puede modificar cualquier parte del sistema.
- Ritmo sostenible de trabajo, evitando la sobrecarga del equipo.
- Cliente in situ, disponible para resolver dudas y priorizar historias.

5.3 XP industrial y el debate XP

En la práctica profesional, pocas organizaciones adoptan XP en su forma "pura": suelen combinar algunas de sus prácticas (integración continua, pruebas automatizadas, programación en pares) con elementos de otros marcos. Este es el llamado "XP industrial".

El debate XP gira en torno a sus límites: ¿es viable en proyectos de gran escala o con equipos distribuidos? ¿Qué ocurre cuando el cliente no puede estar disponible de forma permanente? Estas preguntas son las que la cátedra invita a discutir de forma crítica, y no a repetir el marco como si no tuviera costos ni condiciones de aplicación.

6. Otros modelos ágiles de proceso

6.1 Desarrollo Adaptativo de Software (DAS)

Propuesto por Jim Highsmith, el DAS organiza el ciclo de vida en tres fases: especulación (planificación adaptativa, no predictiva), colaboración (desarrollo concurrente de los componentes, con fuerte comunicación entre los equipos) y aprendizaje (revisión de calidad, del proceso y del trabajo realizado, para retroalimentar el siguiente ciclo). El énfasis del DAS está puesto en la colaboración humana y en aprender del propio proceso, más que en seguir un conjunto fijo de prácticas técnicas.

6.2 Scrum

Scrum es, hoy, el marco ágil más utilizado en la industria. Organiza el trabajo en sprints (iteraciones de duración fija), con roles, eventos y artefactos bien definidos:

Elemento	Descripción
Product Owner	Representa la voz del cliente; define y prioriza el Product Backlog.
Scrum Master	Facilita el proceso, elimina obstáculos y vela por que el equipo respete el marco de trabajo.
Equipo de desarrollo	Grupo autoorganizado y multidisciplinario que construye el incremento en cada sprint.
Sprint	Ciclo de trabajo de duración fija (típicamente 2 a 4 semanas) que produce un incremento de software.
Sprint Planning	Reunión de planificación al inicio del sprint: qué se hará y cómo.
Daily Scrum	Reunión diaria breve de sincronización del equipo (15 minutos aprox.).
Sprint Review	Revisión del incremento construido, con retroalimentación de los interesados.
Sprint Retrospective	Reflexión del equipo sobre el propio proceso, para mejorar el siguiente sprint.
Product Backlog	Lista priorizada y viva de todo lo que el producto podría necesitar.
Sprint Backlog	Subconjunto del Product Backlog seleccionado para el sprint en curso.

7. Modelos prescriptivos vs. procesos ágiles: cuadro comparativo

Este cuadro sintetiza las diferencias centrales entre ambos enfoques, útil como base para que cada grupo argumente su elección de proceso para el proyecto del TPI.

Criterio	Modelos Prescriptivos	Procesos Ágiles
Fases	Secuenciales y predefinidas (análisis, diseño, código, prueba)	Iterativas y superpuestas, en ciclos cortos
Costos	Predecibles al inicio; cambios tardíos son costosos	Se busca mantener bajo el costo del cambio en cualquier momento
Documentación	Extensa y formal, como respaldo del proceso	Mínima e imprescindible; el software funcionando prima sobre la documentación exhaustiva
Comunicación	Formal, basada en documentos y revisiones planificadas	Informal, cara a cara, continua entre el equipo y el cliente
Calendarización	Planificación detallada y de largo plazo	Planificación adaptativa, por iteraciones cortas (sprints)
Seguridad / control	Alta previsibilidad, control por hitos y entregables formales	Alta capacidad de respuesta al cambio; control por inspección y adaptación continua

8. Cierre: ¿qué enfoque conviene para el proyecto del TPI?

No existe una respuesta universal — la elección depende de factores concretos del proyecto de cada grupo:

- Estabilidad de los requerimientos: si el cliente tiene el problema muy claro y no se espera que cambie, un enfoque prescriptivo puede ser razonable.
- Disponibilidad del cliente para retroalimentación continua: sin esa disponibilidad, XP y Scrum pierden buena parte de su efectividad.
- Tamaño y experiencia del equipo: equipos pequeños y autoorganizados se adaptan mejor a marcos ágiles.
- Criticidad y necesidad de documentación formal: proyectos con fuertes exigencias regulatorias suelen requerir más documentación de la que un proceso ágil "puro" contempla.

Esta es exactamente la decisión que cada grupo va a fundamentar en la presentación oral del C3-A01: no se trata de elegir la opción "de moda", sino la que mejor responde a las características reales de su proyecto.